

Dubultaģenta dilemma

- **Laika ierobežojums:** 12 minūtes.
- **Krātuve:** 5 GB
- **Vide:** viens GPU (≈ 16 GB VRAM), bez interneta
- **Risinājuma izmērs:** `solution.ipynb` ≤ 1 MB
- **Bāzlīnijas rezultāts:** 0

Nacionālajā AI centrā Astanā divi datormodeļi — Modelis R (ResNet-18) un Modelis V (ViT-Tiny) — analizē fotogrāfijas. Šobrīd abi modeļi strādā nevainojami, sasniedzot 100% precizitāti un savstarpēji saskaņoti klasificējot katru attēlu. Lai pārbaudītu, cik atšķirīgas patiesībā ir to viedās "smadzenes", galvenais zinātnieks izvirza jums uzdevumu: veiciet niecīgas, gandrīz neredzamas pikseļu izmaiņas katrā fotogrāfijā tā, lai Modelis R un Modelis V pilnībā nesaskanētu.



1. Uzdevums

Divi iepriekš apmācīti attēlu klasifikatori aplūko vienu un to pašu attēlu. Uz šajā uzdevumā sniegtajiem attēliem abi klasifikatori darbojas ar 100% precizitāti.

- **Modelis R:** `torchvision.models.resnet18` (CNN, ResNet18).
- **Modelis V:** `timm vit_tiny_patch16_224` (Transformer, ViT-Tiny).

Jūsu uzdevums ir izveidot nelielu izmaiņu ("perturbāciju") katram attēlam tā, lai abi modeļi nesaskanētu. Katram attēlam jums jāizveido **divas dažādas** perturbācijas:

- **A tips:** pēc tās pievienošanas Modelis R attēlu joprojām klasificē pareizi, bet Modelis V to klasificē nepareizi.
- **B tips:** pēc tās pievienošanas Modelis V attēlu joprojām klasificē pareizi, bet Modelis R to klasificē nepareizi.

Katrai perturbācijai jābūt pietiekami *mazai*, lai to būtu grūti pamanīt. Mazākas perturbācijas dod augstāku punktu skaitu (skatiet 5. sadaļu). Perturbācija tiek pielietota oriģinālajam attēlam tieši pikseļu līmenī.

2. Publiskie dati

Kopā ar uzdevumu tiek sniegta attēlu kopa, kas sadalīta divās daļās — `train` (100 attēli) un `test_public` (100 attēli) — katrā ir attēli ar dažādu izšķirtspēju. Visi attēli ir no ImageNet-1K 1000 klasēm, un gan Modelis R, gan Modelis V abās daļās sasniedz 100% precizitāti.

Tiek sniegti šādi faili:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

Vērtēšanas laikā jūsu `dataset/test_public/` mape tiek nepamanāmi aizstāta ar divām slēptām attēlu kopām (`test_leaderboard_a` un `test_leaderboard_b`) oficiālajai vērtēšanai. Katrā no tām ir **100 attēli** PNG formātā un etiķešu (labels) fails.

Ievērojiet: šajā uzdevumā testa datu kopu etiķetes ir pieejamas.

3. Izvades formāts

Katram attēlam jums jāizveido divi faili:

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- `{index}` (0, 1, 2, ...) atbilst attēla nosaukumam datu kopās.
- Katrs fails ir viens tensors, kas saglabāts ar `torch.save`. Tā formai jābūt $3 \times H \times W$, kur H un W atbilst attiecīgā attēla **oriģinālajai** izšķirtspējai (nevis 224×224).
- Kodam jāizveido tikai viens ZIP fails — `submission.zip`. Ievietojiet visus `.pt` failus ZIP arhīva augšējā līmenī, bez ietverošas mapes vai apakšmapēm.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

Piezīmju grāmata brīdinās jūs, ja būs kādas problēmas ar izvades formātu.

4. Ierobežojumi

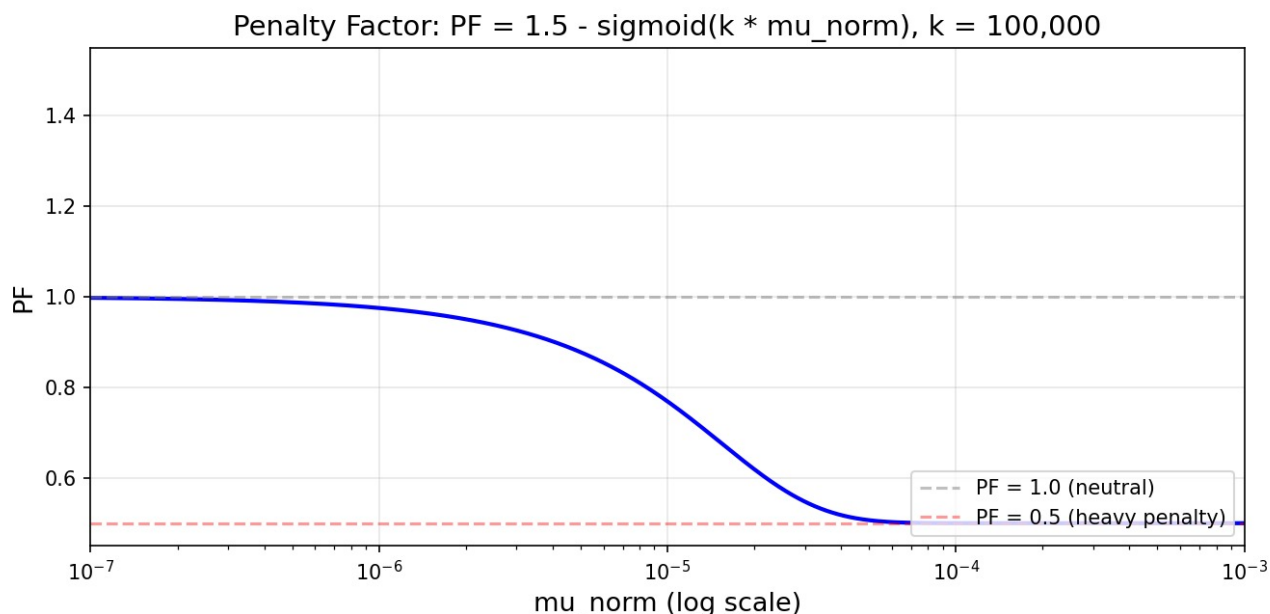
- **Modeļi:** Jums jāizmanto `torchvision.models.resnet18(pretrained=True)` un `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Citi iepriekš apmācīti modeļi nav atļauti.
- **Transformāciju konveijers (piemērots vērtēšanas laikā):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` sīkākai informācijai.
- **Perturbācijas izšķirtspēja:** Tai jāatbilst **oriģinālā** neapstrādātā attēla izšķirtspējai (nevis 224×224). Tensori tiek pievienoti neapstrādātajam attēlam *pirms* transformāciju konveijera.
- **Izvades formāts:** tikai `.pt` faili — nekādu PNG/JPG. Tensori tiek pievienoti neapstrādātajam attēlam, un pikseļu vērtības pirms priekšapstrādes tiek apcirstas uz `[0, 1]`.
- **Failu nosaukumi:** vienā līmenī uzskaitīti, strikti `{index}_a.pt / {index}_b.pt` formātā. ZIP arhīvā nedrīkst būt apakšmapes.
- **Bibliotēkas:** `torch`, `torchvision`, `timm`.

5. Vērtēšana

Gala rezultāts tiek aprēķināts šādi. Lai M ir attēlu skaits daļā, $Score_A$ — veiksmīgo A tipa perturbāciju skaits un $Score_B$ — veiksmīgo B tipa perturbāciju skaits:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF ir funkcija, kas veidota tā, lai sodītu perturbācijas ar lielu normu un būtu ļoti jutīga tuvu veikspējas augšējai robežai. Tā ir ierobežota diapazonā no 0.5 līdz 1. Pilnu implementāciju var apskatīt `solution.ipynb` 8. sadaļā.



Attēls: soda funkcijas līkne.

6. Iesūtījuma pārbaude

Piezīmju grāmatā ir pārbaudes, kas brīdina jūs par formatēšanas problēmām, — `solution.ipynb` piezīmju grāmatas 7. sadaļā.

7. Lokālā testēšana

`solution.ipynb` satur pilnīgu, strādājošu piemēru. Tas ielādē publiskos datus, abus modeļus un oficiālo vērtētāju, kā arī izveido iesūtījuma ZIP failu. Izlasiet to pirms darba sākšanas.

8. Kā iesūtīt

- Saglabājiēt savas izmaiņas failā `solution.ipynb`.
- Atveriet cilni Git JupyterLab kreisajā sānjoslā.
- **Sagatavojiet (stage)** `solution.ipynb` (blakus esošā + ikona).
- Ievadiet commit ziņojumu un noklikšķiniet **Commit**.
- Noklikšķiniet uz mākoņa ar augšupvērstu bultu, lai veiktu push.
- Atgriezieties šajā sacensību (Contest) lapā un noklikšķiniet **Submit**.

Iesūtiet tieši vienu failu ar nosaukumu `solution.ipynb`.